VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY

HO CHI MINH CITY OF TECHNOLOGY

FACULTY OF COMPUTER SCIENCE AND ENGINEERING

PROGRAMMING INTEGRATION PROJECT
SOFTWARE ENGINEERING
(CO3103)

Volunteer and Charity Platform for HCMUT Students

Group 4

Instructor: PhD. Lê Xuân Bách

Full Names	Student ID	Nickname
Lâm Minh Tùng Quân	**2352991**	**4A**
Nguyễn Hoàng Khang	2352479	4B
Nguyễn Minh Quang	2352972	4C
Nguyễn Hoàng Quân	2252680	4D
Nguyễn Minh Quân	2353005	4E

Ho Chi Minh City, December 2025

Final Report

GitHub Repository [Group 4 - GitHub Repository for MVP]

1. Overview

The Volunteer and Charity Platform is a web-based application developed to serve as a centralized intermediary between Ho Chi Minh City University of Technology (HCMUT), affiliated organizations, and the student body. Currently, information regarding volunteer programs and community service activities at HCMUT is fragmented across various channels such as social media platforms, student clubs' pages, and word-of-mouth communication. This fragmentation creates significant barriers for students attempting to access reliable, up-to-date information about volunteer opportunities and for organizers struggling to ensure equitable outreach and efficient participant management. The system is designed with strict Role-Based Access Control (RBAC), securing the platform via **HCMUT_SSO** integration to distinguish between Volunteers (Students), Organizers (Staff), and Administrators.

Project Objectives

The primary objective of the platform is to bridge the gap between the university's social mission and student participation by delivering:

- **A Centralized Information Hub:** Establishing a single official channel for advertising all volunteer and social-work activities, replacing scattered communication methods.
- **Enhanced Visibility and Accessibility:** Ensuring students have real-time access to event schedules, locations, and updates.
- **Academic Integration:** Facilitating the tracking of "Social Work Days," a prerequisite for students to qualify for graduation and senior projects.

List of Main Features

- **User Authentication and Profile Management**: The system enables students and organizers to securely register and log in. Users can update their personal information, and students can synchronize their profiles with school records to track and manage their required social work days.
- **Event Discovery and Search**: The platform provides an intuitive interface for browsing ongoing and upcoming events. Users can efficiently locate events of interest through a keyword-based search function.
- **Event Management**: Organizers are provided with a dedicated management interface that supports creating new events and updating event details as needed
- **Event Registration and Participation Tracking for Students**: Students can register for available events, track the status of their registrations and participation outcomes,

and automatically update their accumulated social service hours based on attendance records.

System Utilities

- **Notification Service:** Automated email and push notifications keep users informed of application results and event updates.
- **Advanced Security:** The platform implements a three-token mechanism (Access, Refresh, and CSRF tokens) to ensure stateless, secure authentication and protection against common web vulnerabilities.

2. Individual Contributions

Member (Nickname)	Contributions to Date
4A	Backend lead; API gateway; JWT + CSRF security; Neon DB integration; database schema
4B	Backend folder structure; Authentication flow; authorization rules; documentation; implement simple backend testing; deployment implementation with CI/CD pipeline
4C	Implementing backend APIs, for Student, Organizer, application management, event management, student tracking activity
4D	Organizer UI components; UI/UX improvement; state-chart diagram
4E	Profile and points tracking; state-chart diagram; responsive design; volunteer UI components

3. System Design and Implementation

System Design

The system is built on a three-layer architecture consisting of a modern frontend, a modular backend, and a cloud-hosted database. The frontend uses React with Tailwind CSS to deliver a fast, responsive user interface, with React Router handling navigation and Axios managing API communication. The backend is implemented with FastAPI, providing high-performance RESTful services, structured modular controllers, and a secure authentication flow using Access, Refresh, and CSRF tokens stored in HttpOnly cookies.

The frontend and backend communicate through a token-secured REST API, where Axios interceptors automatically attach CSRF headers and handle silent token refresh. Data persistence is managed by PostgreSQL (Neon Cloud), which stores core entities such as users, events, slots, applications, and notifications.

All components are containerized with Docker and deployed via a **CI/CD pipeline on AWS**, ensuring consistency, scalability, and streamlined updates.

a. **Backend (Server-side)**

Component	Technology	Details
Framework	FastAPI	Python 3.11+
Database	PostgreSQL 15	Use psycopg2 synchronous driver
Authentication	JWT & OAuth2	Managed via python-jose. Tokens stored in HttpOnly Cookies
Security	Multi-layer	CSRF (Session middleware), Bcrypt hashing, Secure Cookies (HttpOnly, SameSite=Lax)
Documentation	OpenAPI	Auto-generated Swagger UI at /docs

b. **Frontend (Client-side)**

Component	Technology	Details
Framework	React 19.1	React Router DOM 7.9 for routing
Build Tool	Vite 5.0	Optimized build and dev server
HTTP Client	Axios 1.13	Integrated with intercepts for request/response handling
Security	Tailwind CSS 3.4	Utility-first CSS framework
State Mgmt	Context API	AuthContext for global authentication state

Database Schema

Key entities include: USERS, STUDENTS, ORGANIZER, EVENTS, which are responsible for authentication and event management features. EVENT_SLOTS is used for containing the information of timeslots of each event. APPLICATIONS represents the number of applied forms from students when they register to an event. AUDIT LOGS, CONSENTS and NOTIFICATIONS are used for system utilities.

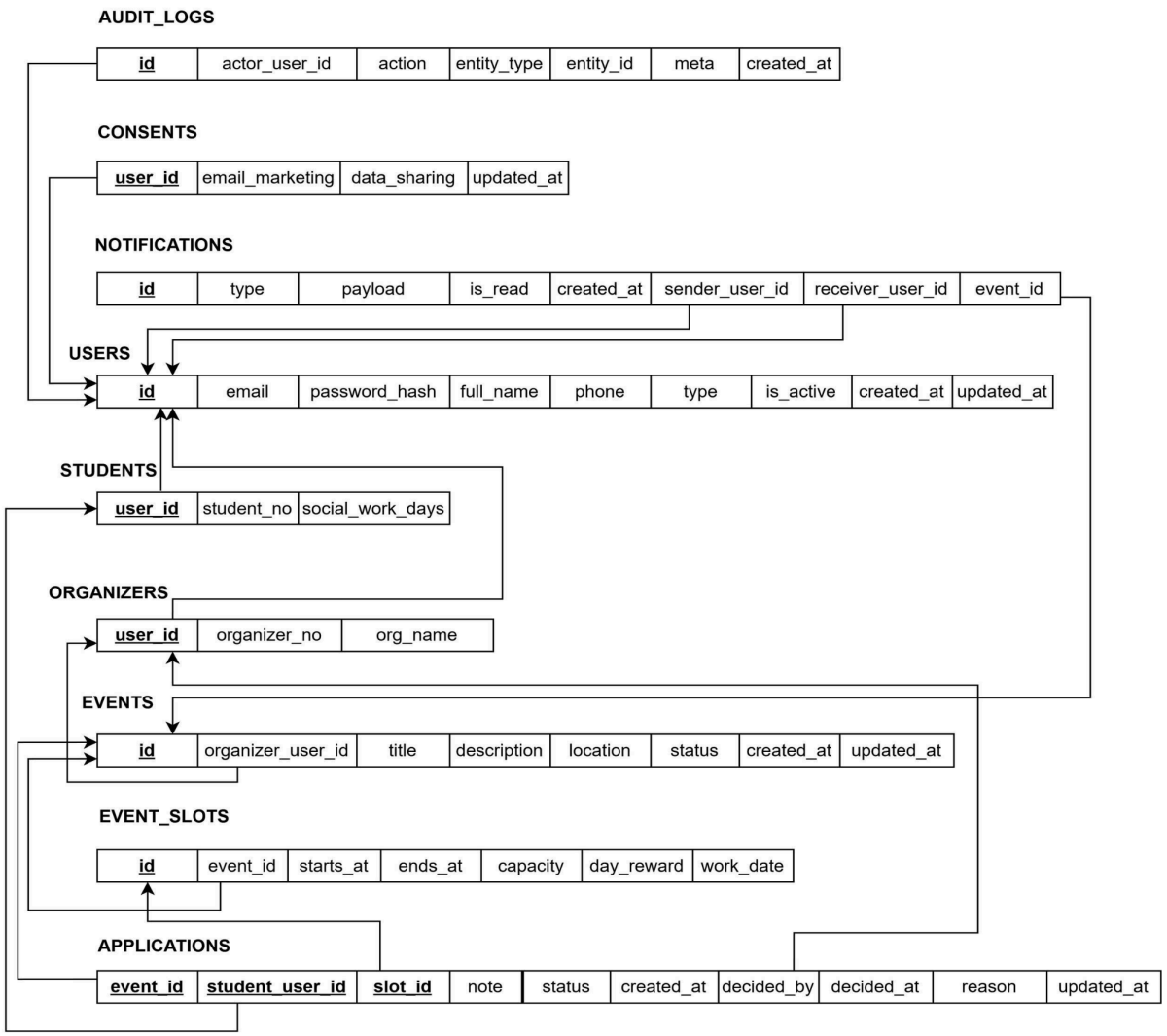


Figure 1. Database Schema

UI & Screen Flow

- ***For Student Role:***

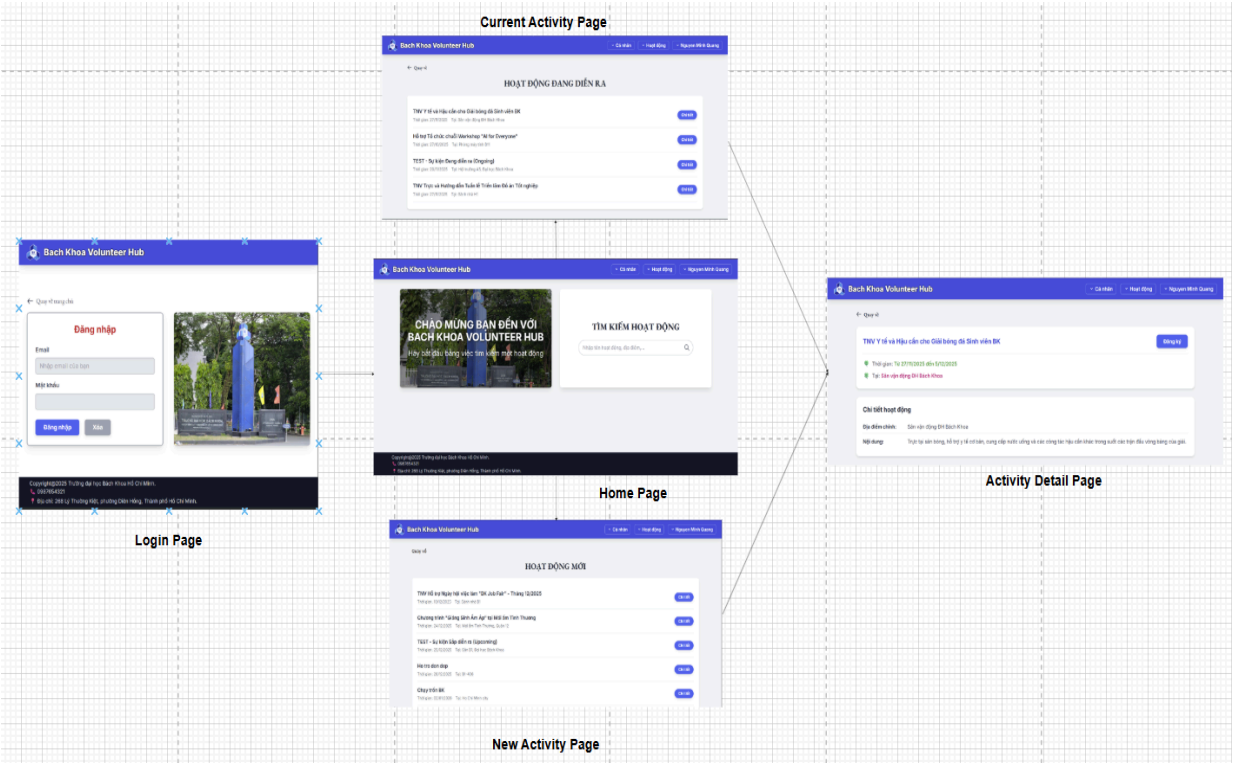


Figure 2: The main flow of the application

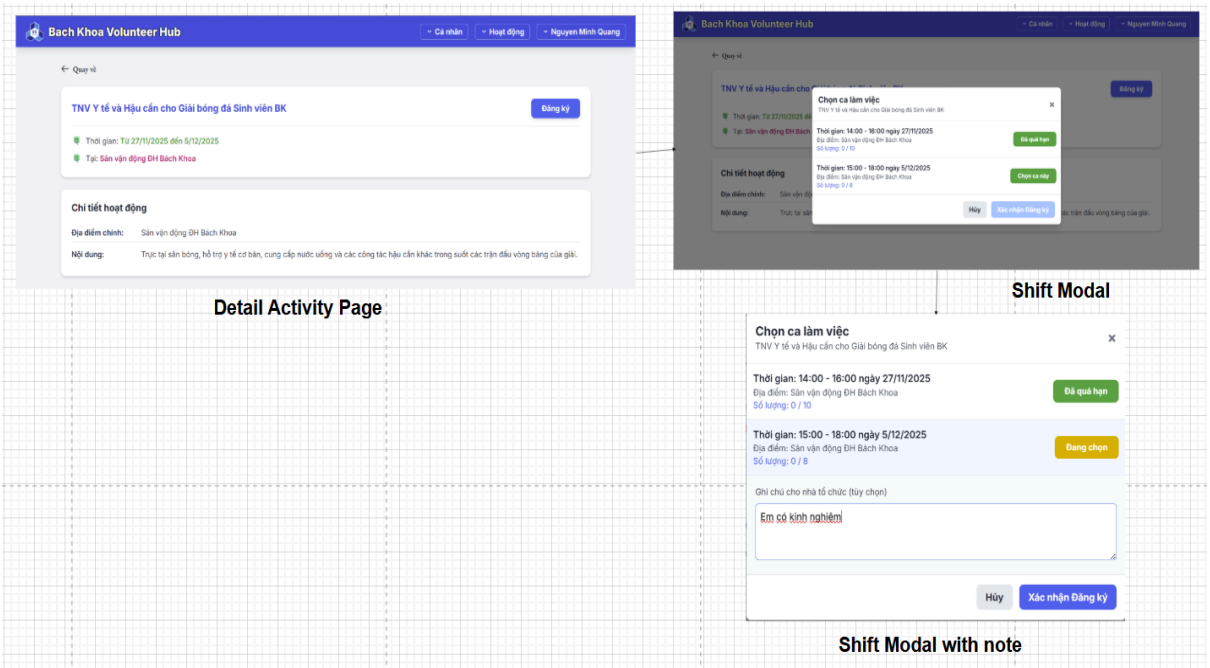


Figure 3: Applying a shift when the activity is still available.

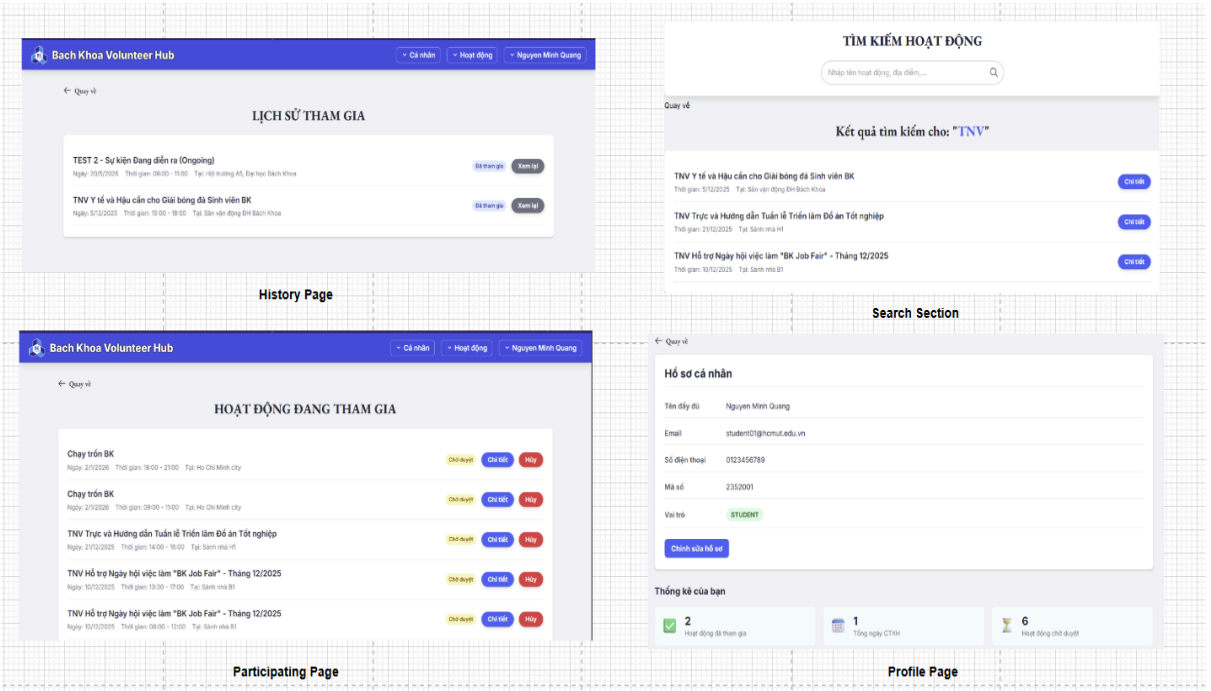


Figure 4: Some other pages supporting different actions.

- Our application utilizes react-router-dom for a single-page experience. Navigation is dictated by the user's authentication status.
- **Unauthenticated Flow**: New users land on the home page. Attempting to access any protected area redirects them to */login*, while remembering their intended destination. After a successful login, they are automatically navigated back to that page or to the main dashboard.
- **Authenticated Flow**: Logged-in students can navigate between pages without re-authentication. The "Activities" menu leads to list pages (*/activities/new, /activities/current*). Clicking "Details" on an item programmatically navigates to a dynamic route like */activities/:id.* The "Personal" menu links to role-protected routes like */participating* and */history*. These pages fetch and display user-specific data from the API. The student can also view their profile and search for activities through */profile* and */search*.
- **Session Persistence**: On page reload, the application automatically re-authenticates the user by verifying the session cookie with the backend, ensuring a persistent login state.

Use-case Diagram

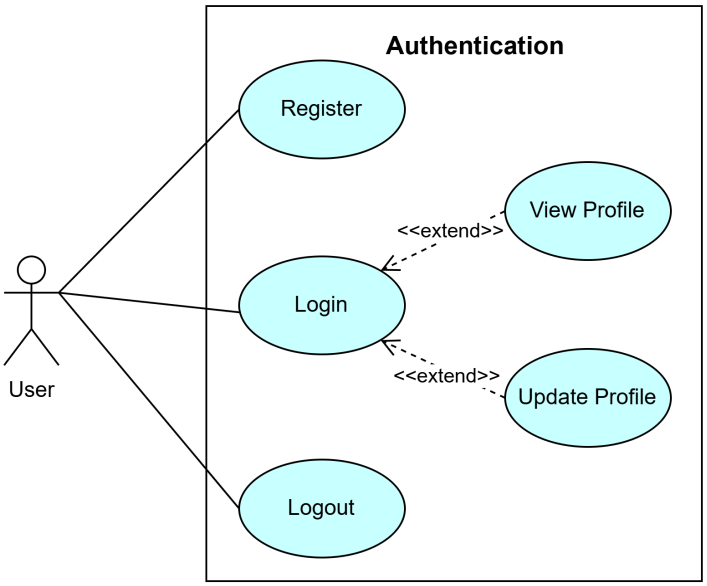


Figure 5. Authentication

The Authentication - diagram illustrates the authentication functionalities available to users, including registering, logging in, and logging out. After logging in, users can extend their actions to view or update their personal profiles.

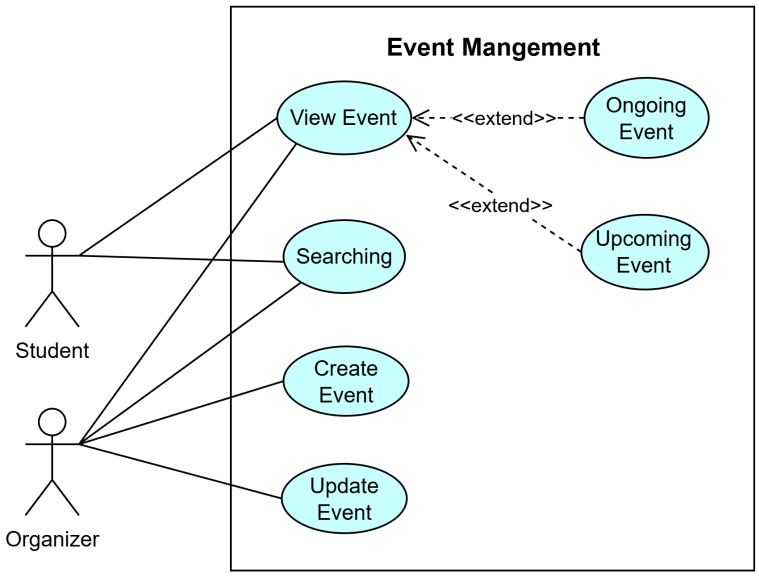


Figure 6. Event Management

The Event Management - diagram represents the event-related functionalities of the system. Users can view events, search for events, and explore both ongoing and upcoming ones through extension use cases. For organizers, the system also supports creating and updating event information.

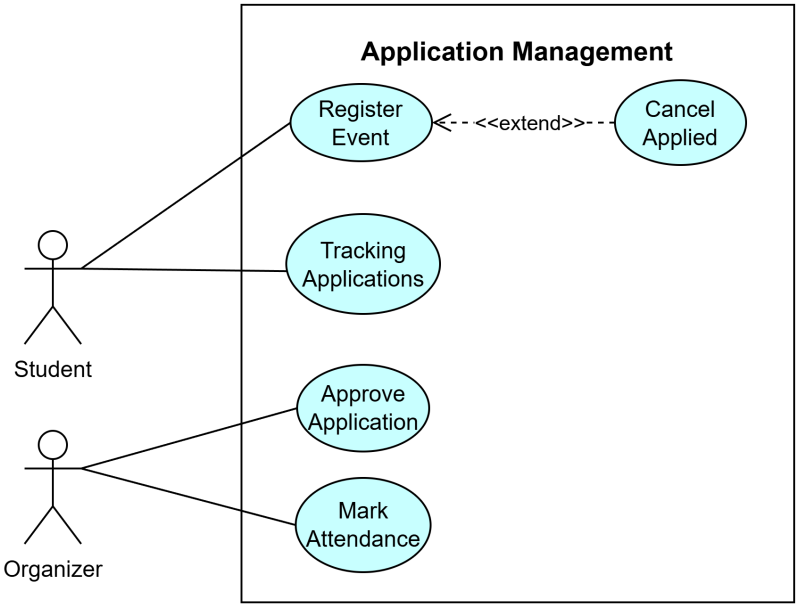


Figure 7. Application Management

The Application Management - diagram outlines how students and organizers interact with event applications. Students can register for events, cancel their applications, and track the status of their submissions. Organizers can approve applications and mark attendance, supporting the workflow required to manage participation and social service hour tracking.

4. Deployment Implementation

To ensure scalability, consistency, and automated delivery, the system utilizes a containerized architecture deployed on Amazon Web Services (AWS). The deployment strategy leverages Docker for containerization and GitHub Actions for Continuous Integration and Continuous Deployment (CI/CD), enabling a seamless transition from development to production.

Infrastructure Architecture

The production environment is hosted on an AWS EC2 instance (Ubuntu Server), serving as the host for the Docker runtime. The system architecture consists of:

- Container Registry: AWS ECR (Elastic Container Registry) is used to securely store versioned Docker images for both the Backend (FastAPI) and Frontend (React/Vite).
- Orchestration: Docker Compose manages the multi-container application, handling networking between the backend API, frontend client, and the database service.

CI/CD Pipeline Workflow

An automated pipeline was established using GitHub Actions to eliminate manual deployment errors. The workflow consists of the following stages:

- Trigger: The pipeline initiates automatically upon a code push to the main branch.
- Build & Push: The system builds Docker images from the source code and pushes them to the private AWS ECR repositories.

- Deploy: The pipeline connects to the EC2 server via SSH, pulls the latest images from ECR, updates the configuration, and restarts the containers using Docker Compose.

This automated process ensures that the live application always reflects the latest stable codebase with minimal downtime.

5. Testing & Quality Assurance

To ensure the reliability and robustness of the Volunteer Platform, a comprehensive testing strategy was implemented, focusing primarily on backend integration testing. The testing framework utilized **Pytest** and **FastAPI TestClient**, covering critical business logic, authentication flows, and data integrity.

Test Plan

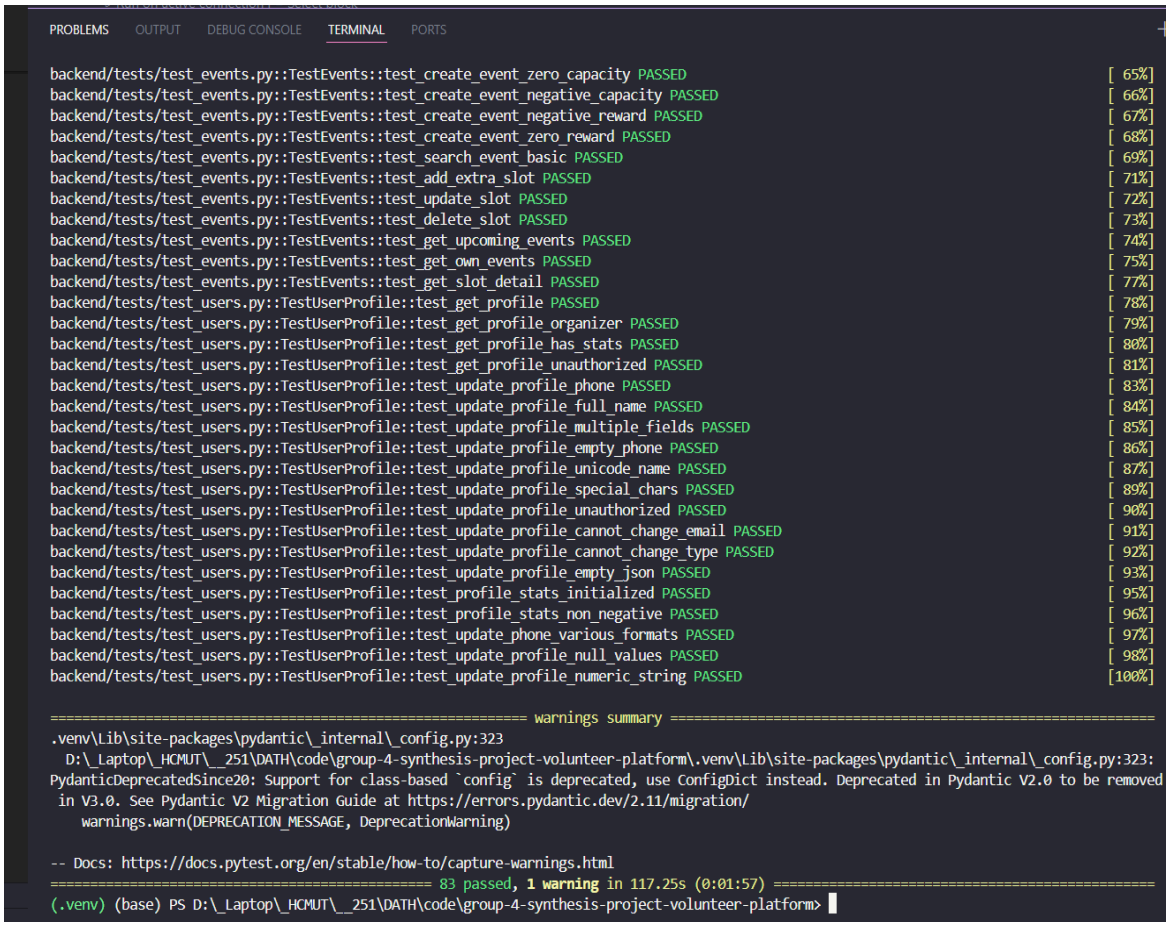
The testing strategy was designed to validate the system's core functionalities under various scenarios:

- **Authentication & Security:** Verifying registration, login, logout, CSRF token validation, and JWT handling.
- **Event Management:** Ensuring organizers can create, update, and manage events and slots correctly.
- **Application Workflow (Core Flow):** Validating the complete lifecycle of a volunteer application: *Apply → Review (Approve/Reject) → Attendance Marking*.
- **User Profile & Statistics:** Confirming that user data retrieval and statistical calculations (e.g., social work days) are accurate.
- **Edge Cases:** Testing invalid inputs, unauthorized access attempts, SQL injection prevention, and boundary conditions (e.g., full capacity slots).

Test Execution & Results

A total of **83 test cases** were executed, achieving a high pass rate and confirming the stability of the API endpoints.

- **Tools Used:** Pytest, HTTPX.
- **Scope:** All major API endpoints (Auth, Events, Applications, Users).
- **Result Summary:**
    - **Passed:** 83 tests
    - **Failed:** 0 tests



```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

backend/tests/test_events.py::TestEvents::test_create_event_zero_capacity PASSED                    [ 65%]
backend/tests/test_events.py::TestEvents::test_create_event_negative_capacity PASSED                [ 66%]
backend/tests/test_events.py::TestEvents::test_create_event_negative_reward PASSED                  [ 67%]
backend/tests/test_events.py::TestEvents::test_create_event_zero_reward PASSED                      [ 68%]
backend/tests/test_events.py::TestEvents::test_search_event_basic PASSED                            [ 69%]
backend/tests/test_events.py::TestEvents::test_add_extra_slot PASSED                                [ 71%]
backend/tests/test_events.py::TestEvents::test_update_slot PASSED                                   [ 72%]
backend/tests/test_events.py::TestEvents::test_delete_slot PASSED                                   [ 73%]
backend/tests/test_events.py::TestEvents::test_get_upcoming_events PASSED                           [ 74%]
backend/tests/test_events.py::TestEvents::test_get_own_events PASSED                                [ 75%]
backend/tests/test_events.py::TestEvents::test_get_slot_detail PASSED                               [ 77%]
backend/tests/test_users.py::TestUserProfile::test_get_profile PASSED                               [ 78%]
backend/tests/test_users.py::TestUserProfile::test_get_profile_organizer PASSED                     [ 79%]
backend/tests/test_users.py::TestUserProfile::test_get_profile_has_stats PASSED                     [ 80%]
backend/tests/test_users.py::TestUserProfile::test_get_profile_unauthorized PASSED                  [ 81%]
backend/tests/test_users.py::TestUserProfile::test_update_profile_phone PASSED                      [ 83%]
backend/tests/test_users.py::TestUserProfile::test_update_profile_full_name PASSED                  [ 84%]
backend/tests/test_users.py::TestUserProfile::test_update_profile_multiple_fields PASSED            [ 85%]
backend/tests/test_users.py::TestUserProfile::test_update_profile_empty_phone PASSED                [ 86%]
backend/tests/test_users.py::TestUserProfile::test_update_profile_unicode_name PASSED               [ 87%]
backend/tests/test_users.py::TestUserProfile::test_update_profile_special_chars PASSED              [ 89%]
backend/tests/test_users.py::TestUserProfile::test_update_profile_unauthorized PASSED               [ 90%]
backend/tests/test_users.py::TestUserProfile::test_update_profile_cannot_change_email PASSED        [ 91%]
backend/tests/test_users.py::TestUserProfile::test_update_profile_cannot_change_type PASSED         [ 92%]
backend/tests/test_users.py::TestUserProfile::test_update_profile_empty_json PASSED                 [ 93%]
backend/tests/test_users.py::TestUserProfile::test_profile_stats_initialized PASSED                 [ 95%]
backend/tests/test_users.py::TestUserProfile::test_profile_stats_non_negative PASSED                [ 96%]
backend/tests/test_users.py::TestUserProfile::test_update_phone_various_formats PASSED              [ 97%]
backend/tests/test_users.py::TestUserProfile::test_update_profile_null_values PASSED                [ 98%]
backend/tests/test_users.py::TestUserProfile::test_update_profile_numeric_string PASSED             [100%]

=========================================================== warnings summary ===========================================================
.venv\Lib\site-packages\pydantic\_internal\_config.py:323
  D:\_Laptop\_HCMUT\__251\DATH\code\group-4-synthesis-project-volunteer-platform\.venv\Lib\site-packages\pydantic\_internal\_config.py:323:
PydanticDeprecatedSince20: Support for class-based `config` is deprecated, use ConfigDict instead. Deprecated in Pydantic V2.0 to be removed
 in V3.0. See Pydantic V2 Migration Guide at https://errors.pydantic.dev/2.11/migration/
    warnings.warn(DEPRECATION_MESSAGE, DeprecationWarning)

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
============================================= 83 passed, 1 warning in 117.25s (0:01:57) ==============================================
(.venv) (base) PS D:\_Laptop\_HCMUT\__251\DATH\code\group-4-synthesis-project-volunteer-platform>
```

Figure 8. Pytest execution result showing all tests passed

6. Future Work and Plan

- As for the future, our team is shifting focus from core feature development to system optimization and production readiness.
    + <u>Feature Completion:</u> We are finalizing the last remaining modules, specifically the Notification System and Admin Tools, to ensure a comprehensive experience for all user roles.
    + <u>Technical Optimization:</u> To enhance scalability and response times, we are refactoring the API to utilize asynchronous and implementing caching strategies.
    + <u>Deployment & Security:</u> The MVP will be deployed to a production environment (utilizing AWS or Render) via our automated CI/CD pipeline. We will also conduct rigorous security audits and load testing to guarantee system reliability under real-world usage.

Final Deliverables: We are wrapping up the technical documentation and the final project report. Additionally, we are preparing the final presentation slides, ensuring they are concise and visual to address the feedback regarding readability and pacing from the previous review.

7. Project Conclusion and improvements

Current Status & Strengths We have successfully reached a functional **Minimum Viable Product (MVP)**. The platform now features a secure, role-based system distinguishing between Students, Organizers, and Administrators.

- **Robust Testing:** We achieved a 100% pass rate on 83 backend integration tests, covering all major API endpoints.
- **Automated Deployment:** We established a CI/CD pipeline using GitHub Actions and Docker, ensuring that code is automatically built and pushed to AWS ECR.

Addressing Previous Weaknesses We took the feedback from our previous presentation seriously and have made specific improvements in how we document and present the system:

- **Feedback: "Technical diagrams are too dense, making them hard to read."**
    - **Improvement:** In this final report, we moved away from cluttered, all-in-one diagrams. We have broken the system design down into focused, modular Use-Case diagrams:
        - **Authentication Flow** is now isolated to show login/register steps clearly.
        - **Event Management** is separated to focus solely on creating and viewing events.
        - **Application Management** has its own dedicated diagram.
    - This modular approach ensures that the logic is easy to follow without overwhelming the viewer.
- **Feedback: "Slides contain too much text" & "Difficult to follow."**
    - **Improvement:** To improve readability and scannability, we have utilized **structured tables** in our documentation to present technical stacks and API endpoints concisely, rather than writing long paragraphs.
    - For the upcoming final presentation, we are specifically preparing slides that prioritize visuals over text to support a smoother, more paced delivery.

Appendix

1. **Overview for the architecture of the system**

The system follows a **three-layer design**, following:

- Presentation Layer: React + Tailwind for Frontend (Organizer and Student portals)
- Business Logic Layer: FastAPI backend with modular services (Auth, Campaign, Application, Attendance)
- Persistence Layer: PostgreSQL (using Neon Cloud)

Frontend: REST/JSON → Backend: [API Layer] → [Service Layer]

├─ In Memory Repository

└─ MySQL

2. **Use-case Diagram Details**

Use-case diagram for whole system and use-case scenario details are in following link:
Use-cases Details

3. **Database schema description**

The database schema of *Volunteer Platform* is designed to manage the complete volunteer activity lifecycle at HCMUT. The schema consists of four main functional modules with cores tables of each module, following:

a. **User Management**

- **USER**
  - Primary Key: id
  - Other attributes: email, password_hash, full_name, phone, type, is_active, created_at, updated_at
  - Use-for: stores information for all system users
- **STUDENT**
  - Primary Key: user_id
  - Other attributes: student_no, social_work_days
  - Use-for: stores information of students participating in volunteer platform
- **ORGANIZER**
  - Primary Key: user_id
  - Other attributes: organizer_no, org_name
  - Use-for: stores information of organizer

b. **Event Management**

- **EVENT**
  - Primary Key: id
  - Other attributes: organizer_user_id, title, description, location, status, created_at, updated_at
  - Use-for: stores information about volunteers campaigns
- **EVENT_SLOT**
  - Primary Key: id

- Other attributes: event_id, starts_at, ends_at, work_date, capacity, day_reward
- Use-for: divides events into specific work shifts for later management

c. Application Management

- **APPLICATION**
- Primary Key: event_id, student_user_id, slot_id
- Other attributes: note, status, created_at, decided_by, decided_at, reason, updated_at
- Use-for: stores the applications of students for each event

d. Supporting Service

- **NOTIFICATION**
- Primary Key: id
- Other attributes: type, payload, is_read, created_at, sender_id, receiver_id
- Use-for: send notifications about application status, new events. reminders

4. API Documentation

Given short description of API endpoints for reviewing, details content and specific for each API endpoint following this link: ĐATH_api_doc.pdf

a. Authentication

Base path: /auth

Endpoint	Method	Context	Description
/register	POST	LoginForm	Create a new user account.
/login	POST	LoginForm	Authenticate and set session cookies.
/logout	POST	Header	Invalidate session and clear cookies.
/csrf	GET	AuthContext	Retrieve/refresh CSRF token.
/refresh	POST	apiClient	Refresh expired access token.

b. User Profile

Base path: /users

Endpoint	Method	Context	Description
/profile/me	GET	AuthContext	Retrieve current user profile and stats.
/profile/me	PUT	ProfilePage	Update personal information.

c. Events & Activities

Base path: /events

Endpoint	**Method**	**Context**	**Description**
/search?q={query}	GET	SearchResults	Search events by keyword.
/get-all-event	GET	Dashboard	Retrieve all events (Organizer view).
/get-own-event	GET	Dashboard	Retrieve events created by current Organizer.
/upcoming	GET	NewActivities	List upcoming published events.
/ongoing	GET	CurrentActivities	List currently active events.
/{id}	GET	ActivityDetail	Retrieve event details and time slots.
/	POST	CreateActivity	Create a new event.
/{id}	PUT	ActivityEditor	Update event details.
/{id}	DELETE	Dashboard	Delete an event.

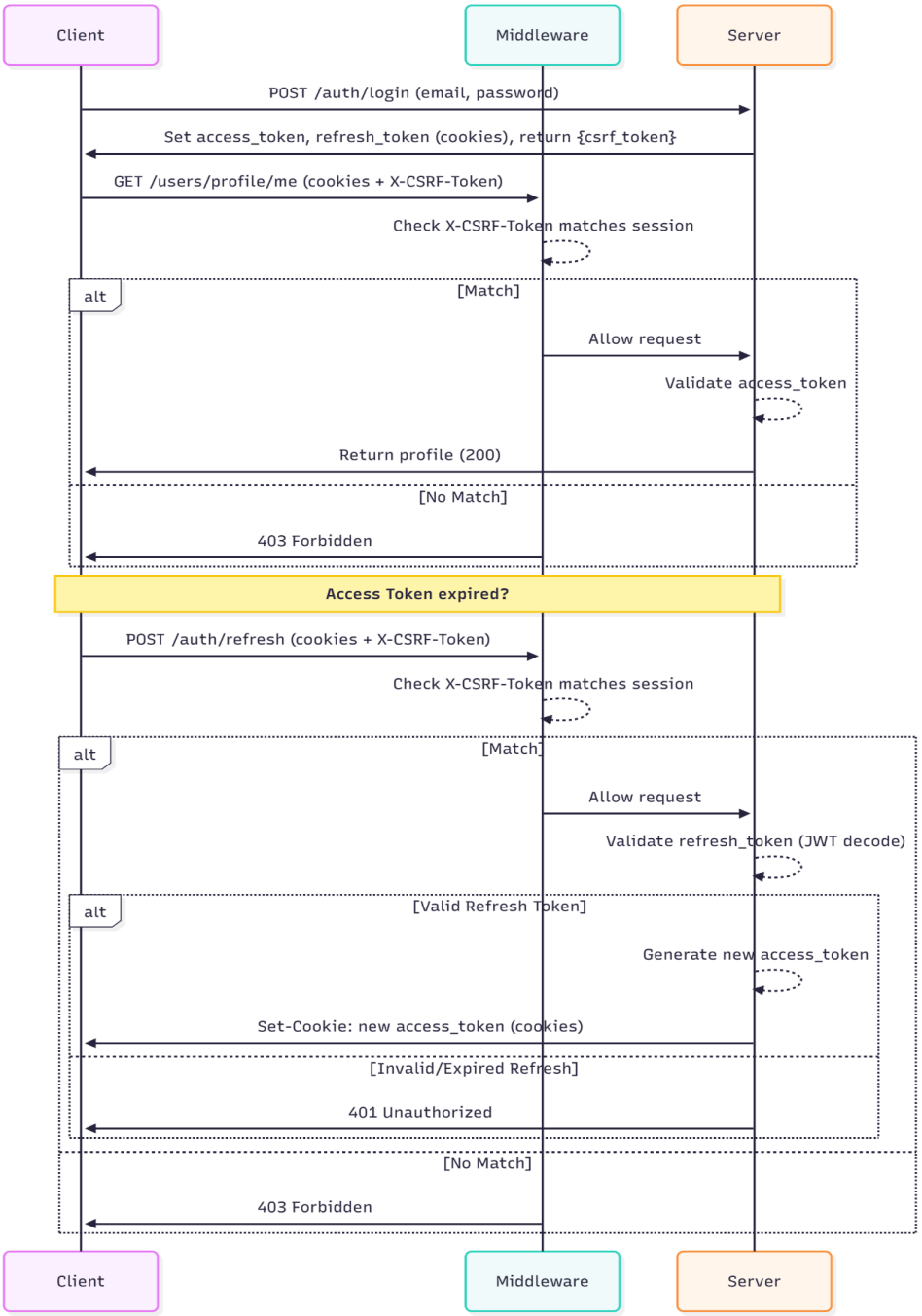
/{id}/slots	POST	CreateActivity	Create time slots for an event.
/slots/{id}	GET	ActivityDetail	Retrieve specific slot details.
/slots/{id}	DELETE	DetailDashboard	Remove a time slot.

d. Applications

Base path: /applications

Endpoint	Method	Context	Description
/apply	POST	ActivityDetail	Student applies for an event slot.
/{id}/review	PATCH	AppReview	Organizer approves/rejects application.
/{id}/attendance	PATCH	DetailDashboard	Mark student attendance.
/{id}/cancel	PATCH	Participating	Student cancels application.
/history	GET	HistoryPage	List past applications.
/participating	GET	Participating	List active applications.
/history/{id}	GET	AppDetail	View past application details.
/participating/{id}	GET	AppDetail	View active application details.
/{event}/slots/{slot}	GET	AppReview	List all applications for a specific slot.

5. Secure Authentication Design



Authentication Flow

a. Registration

Endpoint: POST /auth/register

Process following:

- Frontend: Submits Email, Password, and Account Type (STUDENT/ORGANIZER).
- Backend: Hashes password (Bcrypt), creates User and Role-specific records.
- Response: Returns User Profile (tokens are not issued on register).

b. Login

Endpoint: POST /auth/login

Process following:

- Frontend: Submits credentials.
- Backend: Validates credentials; sets access_token (60 min) and refresh_token (7 days) in HttpOnly cookies; generates CSRF token.
- Frontend: Stores CSRF token in memory via setCsrfToken().

Auth State Initialization

- Goal: Restore session on page reload
- Process following (AuthContext):
    - Calls GET /users/profile/me on mount.
    - If 200 OK: Calls GET /auth/csrf to retrieve a fresh CSRF token; updates state to "Logged In."
    - If 401 Unauthorized: Silently handled as Guest state.

Token Refresh Strategy

- Mechanism: Silent Refresh via Axios Interceptors
- Process following:
    - Interceptor detects 401 Unauthorized response.
    - Calls POST /auth/refresh using the refresh_token cookie.
    - Success: Backend sets a new access_token cookie; Frontend retries the original request.
    - Failure: Redirects user to /login

c. Logout

Endpoint: POST /auth/logout

Process following:

- Backend clears authentication cookies (max_age=0).
- Frontend clears local auth state and CSRF token.
- Redirects to /guest.

6. Detailed Testing Plan

Following information provides a detailed breakdown of the test suites and specific scenarios covered during the quality assurance phase.

Test Environment Setup

- **Framework:** Pytest with FastAPI TestClient.
- **Database:** Tests were executed against a dedicated test database instance to ensure data isolation (or utilizing random data generation to prevent conflicts).
- **Fixtures:** conftest.py was utilized to manage test lifecycles, creating isolated client sessions for Student and Organizer roles to simulate real-world usage without session conflicts.

Detailed Test Scenarios

a. Authentication Module (test_auth.py)

- **Registration:** Verified successful registration for both roles; validated rejection of duplicate emails and missing required fields (e.g., student_no).
- **Login:** Tested valid credentials (returning HTTP 200 & Cookies) and invalid credentials (returning HTTP 401).
- **Security:** Verified that requests without a valid CSRF token in the header are rejected (HTTP 403).

b. Event Management Module (test_events.py)

- **CRUD Operations:** Validated that Organizers can successfully create events with multiple time slots.
- **Authorization:** Confirmed that Students and unauthenticated users cannot create or delete events.
- **Validation:** Tested edge cases such as creating events with past dates, negative capacity, or invalid time ranges.

c. Application Workflow Module (test_applications.py)

- **Full Integration Flow:**
    - The organizer creates an event.
    - Students apply for a specific slot.
    - The organizer approves the application.
    - The organizer marks the student as "Attended".
    - The system updates the student's activity statistics.
- **Constraints:** Verified that students cannot apply to full slots (Capacity limit) or apply twice to the same event.

d. User Profile Module (test_users.py)

- **Data Retrieval:** Ensured users receive the correct profile data and role-specific information.
- **Updates:** Validated that users can update allowed fields (Phone, Full Name) but cannot modify immutable fields (Email, User Type).

Test Code Snippet

For full testing code is implemented in link of GitHub at the beginning of page 2, those following code is an example of an integration test for full application flow:

```
def test_full_application_flow(organizer_auth, student_auth):
    """
    Scenario: Organizer creates event -> Student applies -> Organizer approves -> Organizer
marks attendance.
    """
    org_client, org_headers, _ = organizer_auth
    stu_client, stu_headers, _ = student_auth

    # 1. Organizer creates Event
    res_create = org_client.post("/events/", json=event_payload, headers=org_headers)
    assert res_create.status_code == 201

    # 2. Student applies
    res_apply = stu_client.post("/applications/apply", json=apply_payload,
headers=stu_headers)
    assert res_apply.status_code == 200

    # 3. Verification & Approval
    # ... (assertions for approval and attendance marking) ...
```

7. Detailed Deployment Technical

The following information outlines the technical configurations and scripts used to establish the CI/CD pipeline and production environment.

AWS Environment Configuration

- Compute Resource: AWS EC2 Instance (t3.micro) running Ubuntu 24.04 LTS.
- Security Groups: Inbound rules configured to allow traffic on:
- Port 22 (SSH for administration and GitHub Actions).
- Port 8000 (Backend API).
- Port 5173 (Frontend Application).
- IAM Policy: A dedicated IAM User was created with permissions (AmazonEC2ContainerRegistryFullAccess) to secure the deployment process.

Containerization Strategy

The application is containerized using Dockerfile for each service:

- Backend: Based on python:3.11.6-slim, optimized for minimal image size.
- Frontend: Based on node:22-alpine, utilizing Vite for production builds.
- Database: Utilizing the official postgres:15-alpine image.

CI/CD Workflow Configuration (deploy.yml)

The GitHub Actions workflow automates the delivery process. The core logic implemented in the deployment script could be seen in the deploy.yml file in our group's git repository

Environment Management

Sensitive configurations (Database credentials, JWT Secrets, API Keys) are managed via:

- GitHub Secrets: For CI/CD credentials (AWS Access Keys, SSH Keys).
- Production .env file: Stored securely on the EC2 instance, containing runtime variables such as DATABASE_PASSWORD and API_ORIGIN.